PACE UNIVERSITY

SEIDENBERG SCHOOL OF CSIS

# Model Selection, Model Validation & Boosting

Dr. Krishna Bathula

# Learning Objectives

1. Model Selection Methods
2. Model Validation
3. Cross Validation
4. K-Fold Cross Validation
5. Stratified K-Fold
6. Boosting
7. AdaBoost
8. Gradient Boosting

# Introduction

A **model** is a simplified version of the observations in data.

The simplifications are meant to discard the superfluous details that are unlikely to **generalize to new instances**.

However, one must make assumptions to decide what data to discard and keep.

For example, a **linear model makes the assumption** that the **data is** fundamentally **linear** and that the distance between the instances and the straight line is just **noise**, which can safely be ignored.

If we make absolutely no assumption about the data, then there is no reason to prefer one model over any other. This is called the **No Free Lunch (NFL) theorem**.

# No Free Lunch Theorem

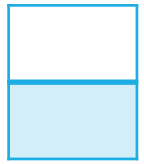
For some datasets, the best model is a **linear model**, while for other datasets it is a **decision tree** or **neural network**.

There is no model that is a priori guaranteed to work better (hence the name of the theorem).

The only way to know for sure which model is best is to **evaluate** them all.

Since this is not possible, in practice, we make some reasonable assumptions about the data, and we evaluate only a few reasonable models.

For example, for **simple tasks**, we may evaluate **linear models**, and for **complex problems**, we may evaluate various other models such as **decision trees** or **neural networks**.



# No Free Lunch Theorem

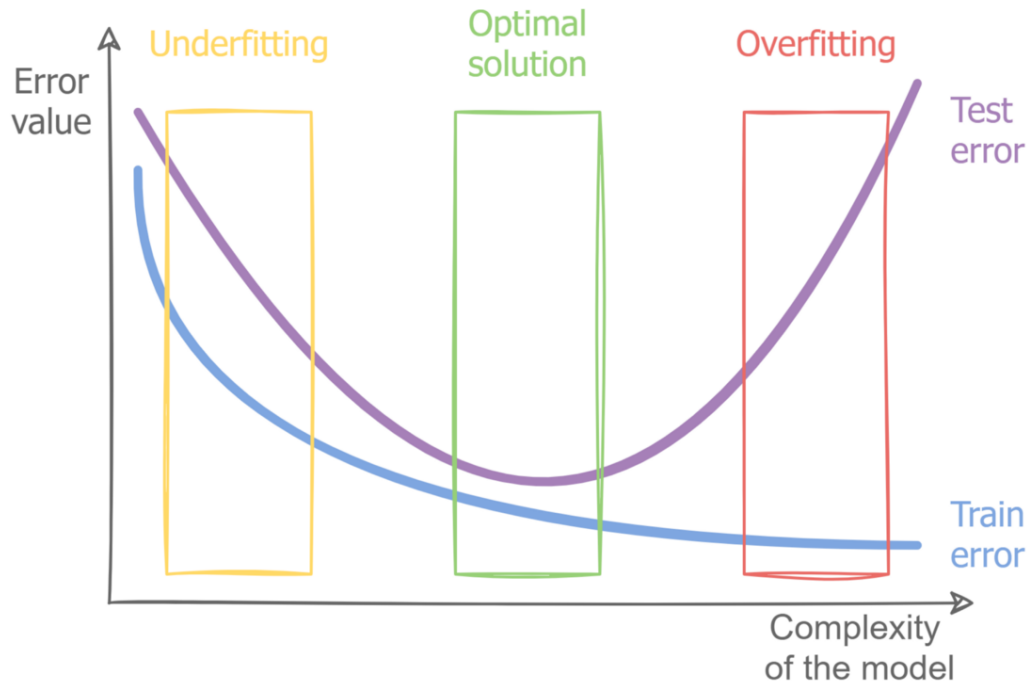
**No single algorithm** will **solve all your machine learning problems** better than every other algorithm.

Make sure we **completely understand a machine learning problem** and the **data** involved before selecting an algorithm to use.

All models are only as good as the assumptions they were created with and the data used to train them.

**Simpler models** like logistic regression have **more bias** and tend to **underfit**, while **more complex models** like neural networks have **more variance** and tend to **overfit**.

# No Free Lunch Theorem




The **best models** for a given problem are somewhere in the middle of the two **bias-variance** extremes.

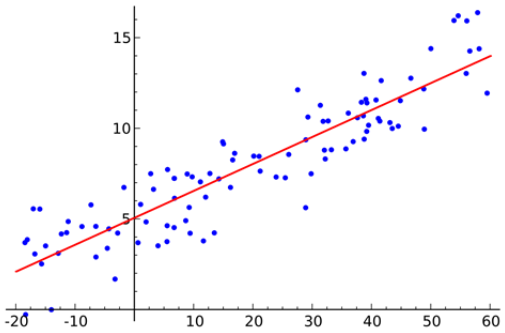
To **find a good model** for a problem, you may have to **try different models** and compare them using a robust **cross-validation strategy**.

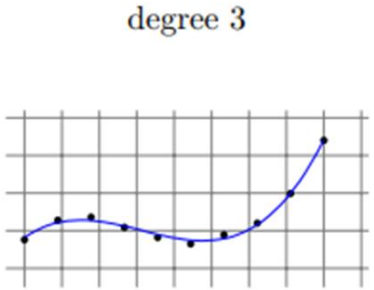
# Model Selection

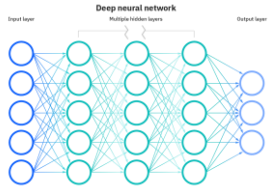
More generally, when approaching some practical problem, we usually can think of **several algorithms** that may yield a **good solution**, each of which might have **several parameters**.

How can we choose the best algorithm for the particular problem at hand?

And how do we set the algorithm's parameters?



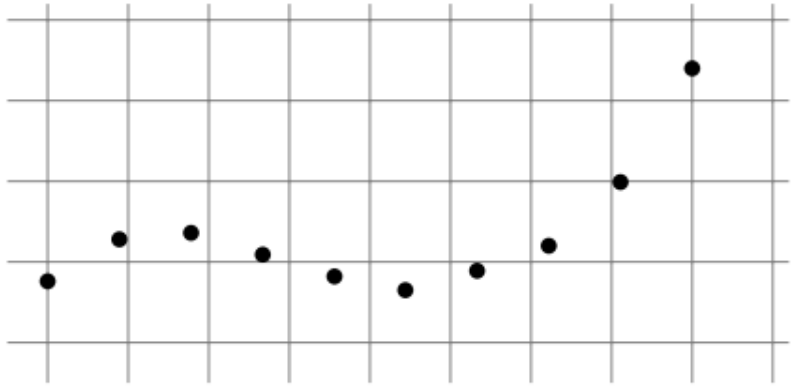




This task is often called model selection.

# Model Selection - Example

To illustrate the model selection task, consider the problem of learning a one dimensional regression function, h : R → R.

Suppose that we obtain a training set as depicted in the figure.

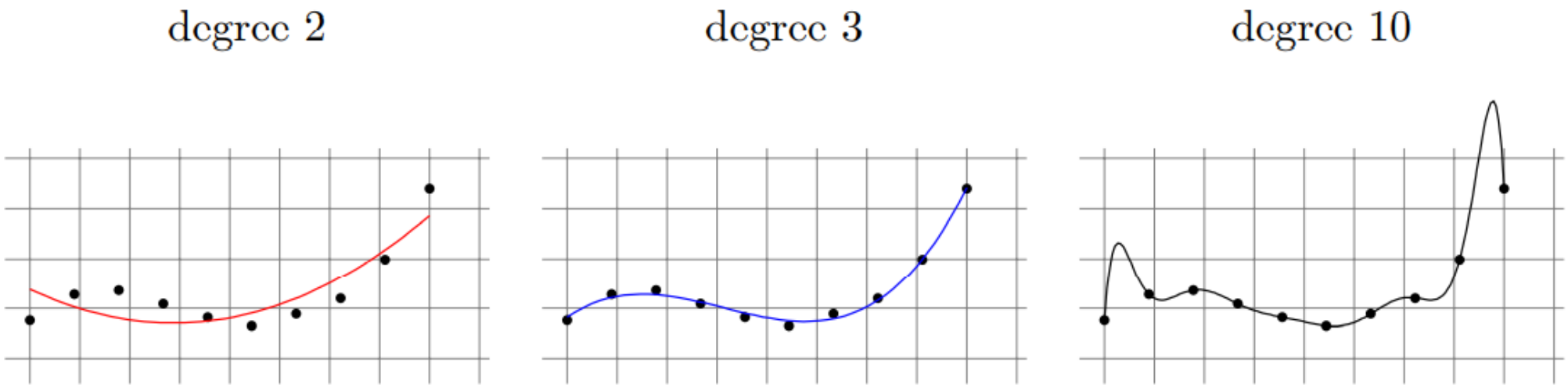


We can consider fitting a **polynomial** to the data

However, we might be uncertain regarding which **degree d** would give the best results for our data set.

# Model Selection – Example (Selecting Degree of Polynomial)

- A **small degree may not fit the data well**. It will have a large training (approximation) error **leading to underfitting**.
- Whereas a **high degree may lead to overfitting** i.e., it will have a large test (estimation) error.
- Following figure depicts the result of fitting a polynomial of degrees **2**, **3**, and **10**.



# Model Selection – Approaches

There are two approaches for model selection.

1. The first approach is based on the Structural Risk Minimization (**SRM**) paradigm.
   - SRM is particularly useful when a learning algorithm depends on a **parameter** that controls the bias-complexity tradeoff.
   - Such as the **degree of the fitted polynomial** in the polynomial regression.

2. The second approach relies on the concept of **validation**.
   - The basic idea is to partition the **training set into two sets**.
   - **One** is used for **training** each of the candidate models, and the **second is used for deciding** which of them yields the best results.

# Model Selection

In model selection tasks, we try to find the right balance between **estimation(training)** error and **approximation(test)** error.

More generally, if our learning algorithm fails to find a predictor with a small risk, it is important to understand whether we suffer from **overfitting** or **underfitting**.

**Feature Analysis**

**Algorithm Selection**

**Parameter Tuning**

# Model Selection – Tradeoffs

**SRM** can be used for tuning the tradeoff between **bias** and **complexity** without deciding on a specific hypothesis class in advance.

- Consider a countable sequence of hypothesis classes $H_1$, $H_2$, $H_3$, . . ..
- For example, in the problem of polynomial regression mentioned, we can take $H_d$ to be the set of polynomials of degree at most d.
- Even though the **empirical risk** of the **$10^{th}$ degree** polynomial is smaller than that of the **$3^{rd}$ degree** polynomial, we would still **prefer the $3^{rd}$ degree** polynomial since its complexity (as reflected by the value of the function g(d)) is much smaller.

**Ex: $\Theta_0 + \Theta_1 x + \Theta_2 x^2 + \Theta_3 x^3$**

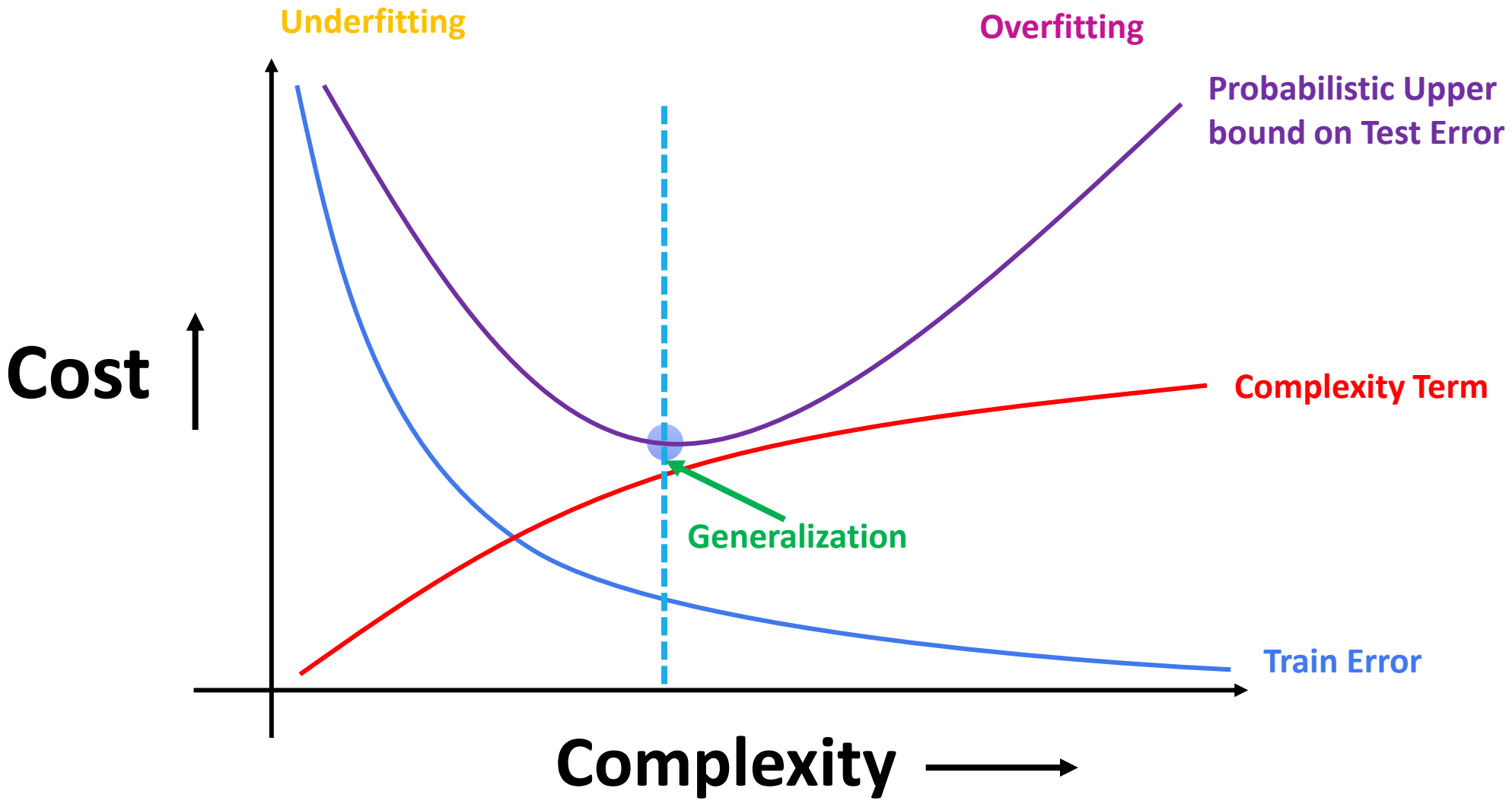
# Data Split

- The data split divides the dataset into 3 parts: a **training** set, a **validation** set(sometimes called **development**), and a **test** set.
- Then we train our model on the training dataset, perform model selection on the validation dataset, and do a final evaluation of the model on the test dataset.
- In this way, the model with the lowest generalization error can be determined.
- **The generalization error refers to the performance of the model on unseen data**, i.e. data that the model hasn't been trained on.

# Validation

- It is always a best practice to get a better **estimation of the true risk** of the output predictor of a learning algorithm.
- One of the strategies was to derive bounds on the estimation error of a hypothesis class. This ensures that for all hypotheses in the class, the **true risk** is not very far from the **empirical risk**. (Recall VC dimension-next slide)
- However, these bounds might be loose and pessimistic, as they hold for all hypotheses and all possible data distributions.
- A more **accurate estimation of the true risk** can be obtained by using some of the training data as a **validation set**, over which one can evaluate the success of the algorithm's output predictor. This procedure is called **Validation**.

# Model Complexity and Cost



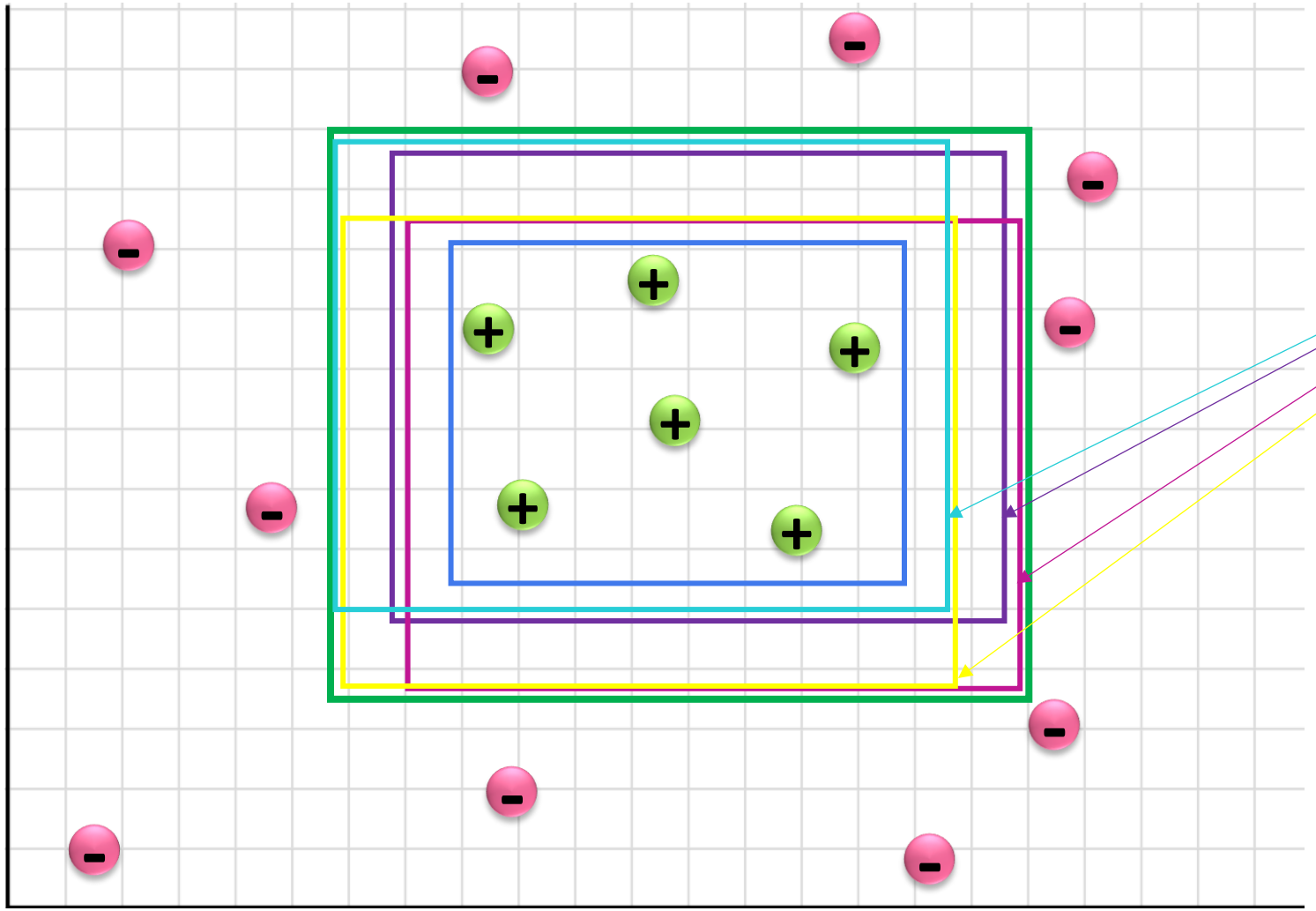
# Validation for Model Selection

- We first train **different algorithms** (or the **same algorithm with different parameters**) on the given training set.
- Let $\mathbf{H = \{h_1, \ldots, h_r\}}$ be the set of all output predictors of the different algorithms.
- In the case of training polynomial regressors, we would have each $\mathbf{h_d}$ be the output of polynomial regression of degree **d**.
- Now, to choose a single predictor from H we sample a **fresh validation set** and choose the predictor that minimizes the error over the validation set.
- In other words, we apply $\mathbf{ERM_H}$ over the **validation set**.

# Limiting bounds of H

- **H** is **not fixed** ahead of time but rather **depends** on the **training set**.
- Since the validation set is independent of the **training set** it **is also independent of H**.
- Therefore, the same technique we used to derive bounds for finite hypothesis classes holds here as well.
- The **error on the validation set approximates the true error** as long as H is not too large.
- If **H is too large** then we are at **risk of overfitting**.

# Validation for Model Selection



**Hypothesis set**
$H = \{h_1, h_2, h_3, h_4 \ldots, h_d\}$

**Selecting one h that has a low ERM over the validation set**

# Train-Validation-Test Split

From the Train, Validation and Test split:

- The **first set** is used for **training our algorithm** and the **second** is used as a **validation set for model selection**.

- After we select the best model, we test the performance of the output predictor **on the third set**, which is often called the "**test set**."

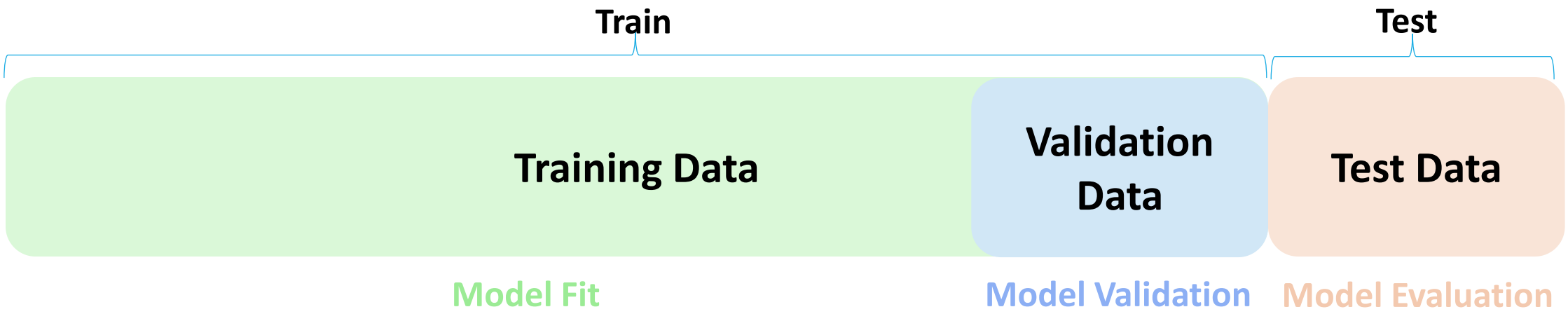
- The number obtained is used as an estimator of the true error of the learned predictor.

- If we only have a small dataset, splitting it into a training and test dataset at a 80:20 ratio respectively doesn't seem to do much

# Hold Out Set

- The simplest way to estimate the true error of a predictor h is by sampling an additional set of examples, **independent of the training set**, and using the **empirical error** on this **validation set as our estimator**.

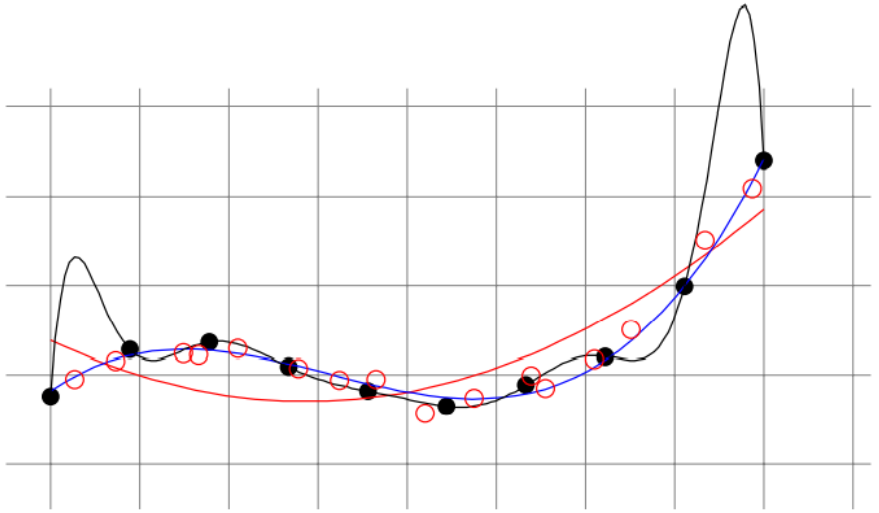
- Sampling a **training set** and then sampling an independent **validation set** is equivalent to randomly partitioning our random set of examples into two parts, using one part for training and the other one for validation.



# Role of Validation Error in Model Selection

consider again the example of fitting a one dimensional polynomial.

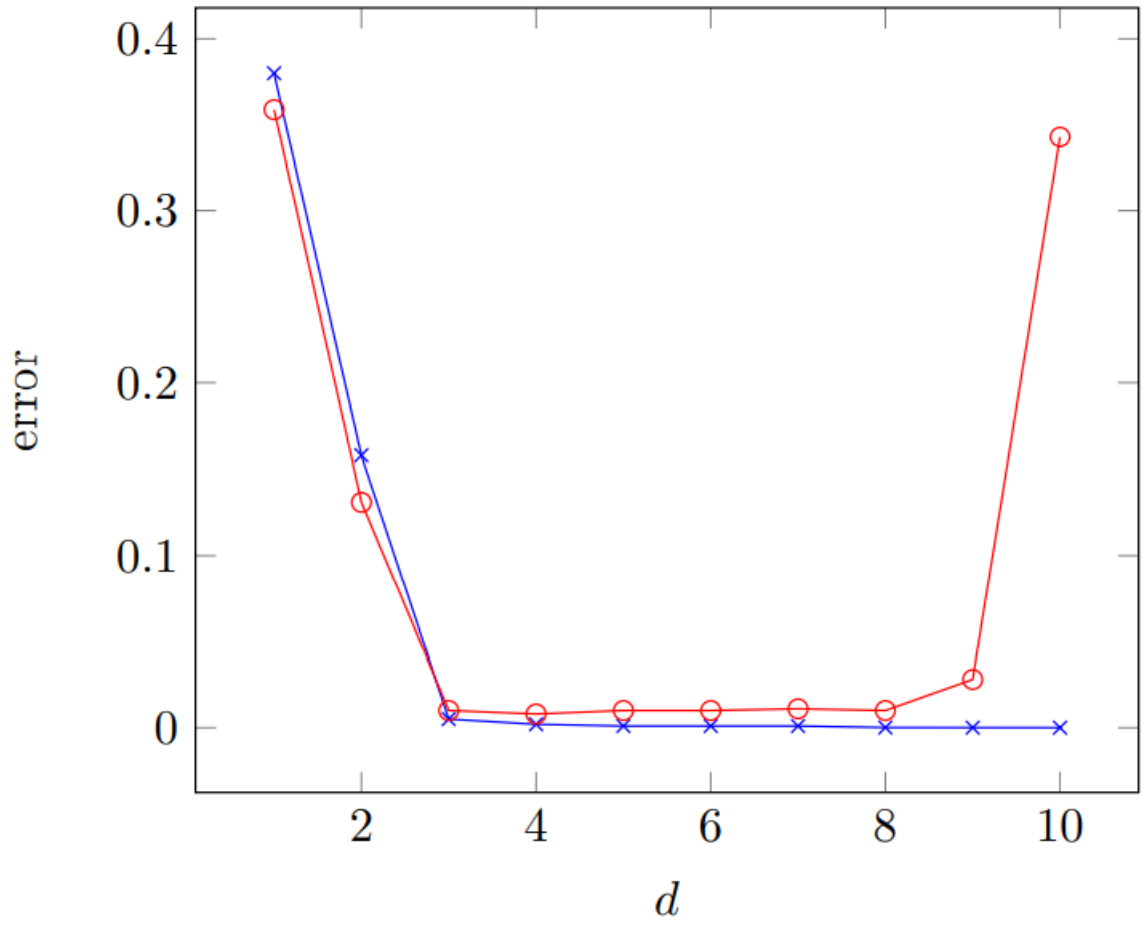
- Using the same training set, with ERM polynomials of degrees **2**, **3**, and **10**, but this time we also depict an additional validation set



- $S_{train}$
- $S_{validation}$

- The polynomial of degree **10 has minimal training error**, yet the polynomial of **degree 3 has minimal validation error**, and hence it will be chosen as the best model.

# Model Selection Curve




d=degree of the polynomial x2 ,x3……

M1 d=1 → $S_{train}$ → $S_{validation}$

M2 d=2 → $S_{train}$ → $S_{validation}$

M3 d=3 → $S_{train}$ → $S_{validation}$

M4 d=10 → $S_{train}$ → $S_{validation}$

The **model selection curve** shows the **training error and validation error** as a **function of the complexity of the model** considered.

# Model Selection Curve – Polynomial Regression

- The **training error** decreases as **the polynomial degree increases** (which is the complexity of the model).
- On the other hand, the **validation error first decreases** but then starts to **increase**, which indicates that the model is starting to suffer from **overfitting**.
- **Plotting such curves** can help us understand whether we are searching for the correct regime of our **parameter space**.
- Often, there may be **more than a single parameter to tune**, and the possible number of values each parameter can take might be quite large.

# Cross Validation

- The overall aim of **Cross-Validation** is to use it as a tool to **evaluate** machine learning models, by training a number of models on **different subsets** of the input data.

- Cross-validation can be used to **detect overfitting** in a model which infers that the model is not effectively generalizing patterns and similarities in the new input data.

- Cross-validation also referred to as **rotation estimation** and/or **out-of-sample testing**.

# Cross Validation -WorkFlow

To perform **cross-validation**, the following steps are typically considered:

1. Split the dataset into **training data** and **test data**
2. The **parameters** will undergo a **Cross-Validation test** to see which are the best parameters to select.
3. These **parameters** will then be implemented into the **model for retraining**
4. Final evaluation will occur and this will depend if the **cycle has to repeat**, **depending on the accuracy** and the level of generalization that the model performs.

# Cross Validation -Types

There are different types of Cross-Validation techniques:

1. **Hold-out**
2. **K-folds**
3. Leave-one-out
4. Leave-p-out

However, we will be mainly focusing on **K-folds**.

# k-Fold Cross Validation

- When using K-fold cross validation, **all parts of the data** will be able to be used as part of the **testing data**.
- This way, all of our data from our **small dataset** can be used for **both training and testing**, allowing us to better evaluate the performance of our model.
- The k-fold cross validation technique is designed to give an **accurate estimate of the true error** without wasting too much data.
- In k-fold cross validation the original training set is **partitioned into k subsets** (folds) of size m/k (for simplicity, assume that m/k is an integer).
- For each fold, the **algorithm is trained on the union of the other folds** and then the error of its output is estimated using the fold.

# k-Fold Cross Validation

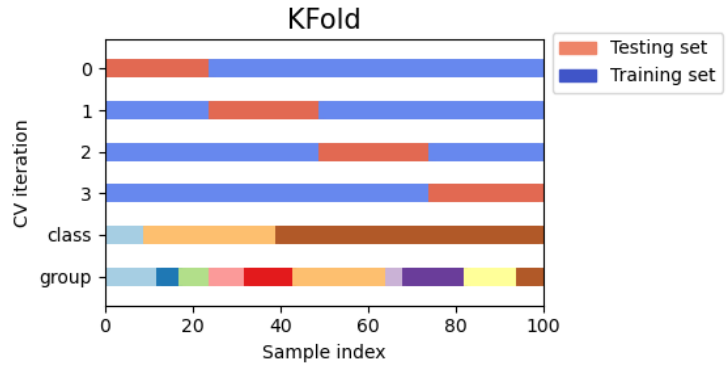


All Data

Training data | Validation data

| | Fold 1 | Fold 2 | Fold 3 | Fold 4 | Fold 5 |
|---|---|---|---|---|---|
| Split 1 | Fold 1 | Fold 2 | Fold 3 | Fold 4 | Fold 5 |
| Split 2 | Fold 1 | Fold 2 | Fold 3 | Fold 4 | Fold 5 |
| Split 3 | Fold 1 | Fold 2 | Fold 3 | Fold 4 | Fold 5 |
| Split 4 | Fold 1 | Fold 2 | Fold 3 | Fold 4 | Fold 5 |
| Split 5 | Fold 1 | Fold 2 | Fold 3 | Fold 4 | Fold 5 |

Finding Parameters




Final evaluation | Test data

# k-Fold Cross Validation - Algorithm

A pseudocode of k-fold cross validation for model selection is given below.

**$k$-Fold Cross Validation for Model Selection**

**input:**
training set $S = (\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_m, y_m)$
set of parameter values $\Theta$
learning algorithm $A$
integer $k$
**partition** $S$ into $S_1, S_2, \ldots, S_k$
**foreach** $\theta \in \Theta$
**for** $i = 1 \ldots k$
$h_{i,\theta} = A(S \setminus S_i; \theta)$
$\text{error}(\theta) = \frac{1}{k} \sum_{i=1}^{k} L_{S_i}(h_{i,\theta})$
**output**
$\theta^\star = \text{argmin}_\theta \, [\text{error}(\theta)]$
$h_{\theta^\star} = A(S; \theta^\star)$

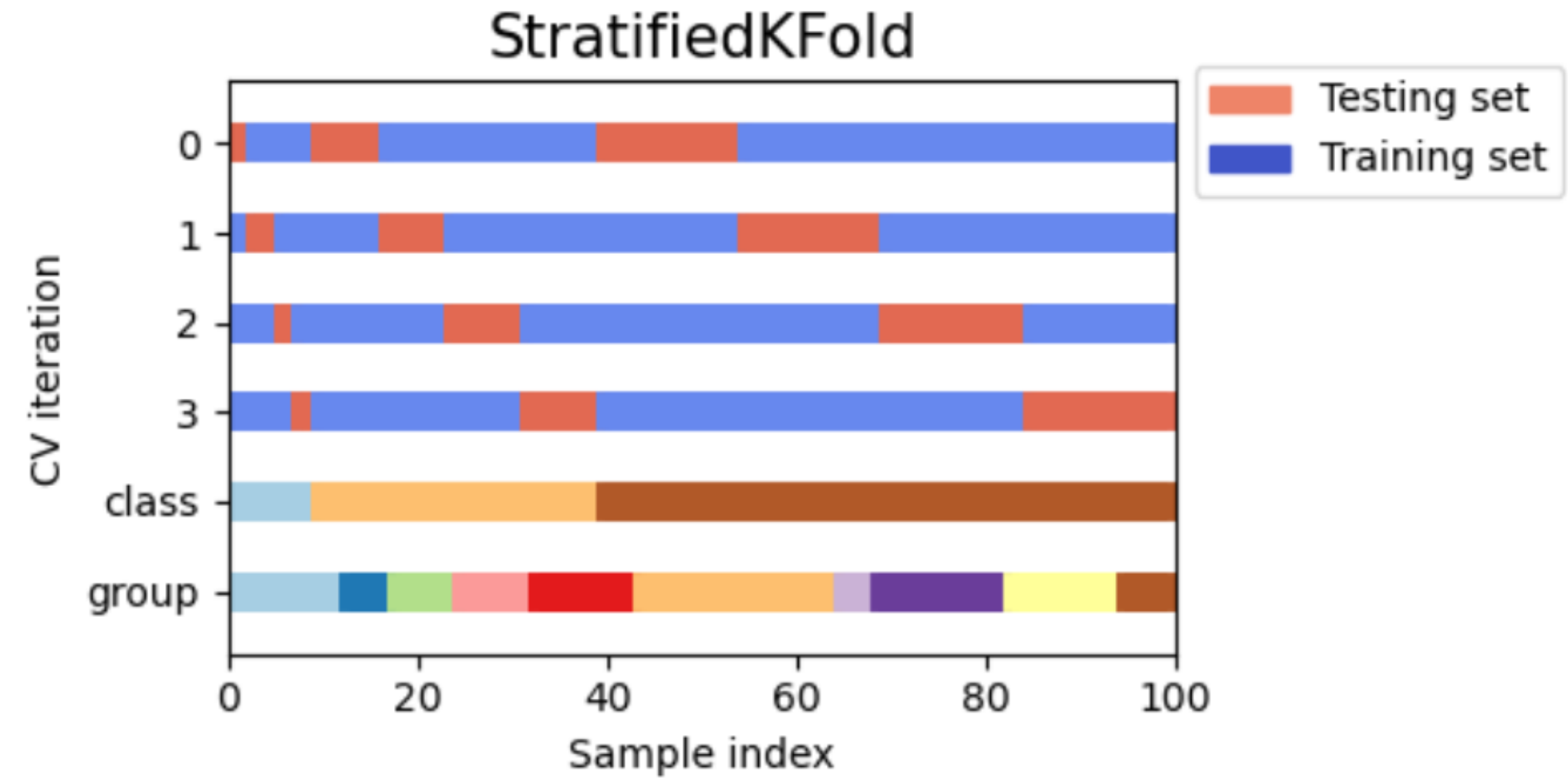
# k-Fold Cross Validation - Limitations

- The Cross Validation method often works very well in practice. However, it might sometime fail
- Rigorously understanding the exact behavior of Cross Validation is still an open problem.
- Other papers show that Cross Validation works for stable algorithms.
- The K Fold Cross Validation approach will **not operate as predicted** for an **unbalanced data set**.
- When a **data set is unstable**, a modest modification to the K Fold cross-validation procedure is required to ensure that each fold has nearly the same number of samples from each output class as the complete.

# Stratified k-Fold

- Some classification problems can exhibit a **large imbalance** in the distribution of the target classes.
- For instance there could be several times **more negative samples than positive samples**.
- In such cases it is recommended to use **Stratified sampling** as implemented in **StratifiedKFold** and **StratifiedShuffleSplit** to ensure that relative class frequencies is approximately preserved in each train and validation fold.
- **StratifiedKFold** is a variation of k-fold that returns stratified folds.
- Each set contains approximately the **same percentage of samples** of each **target class** as the complete set.

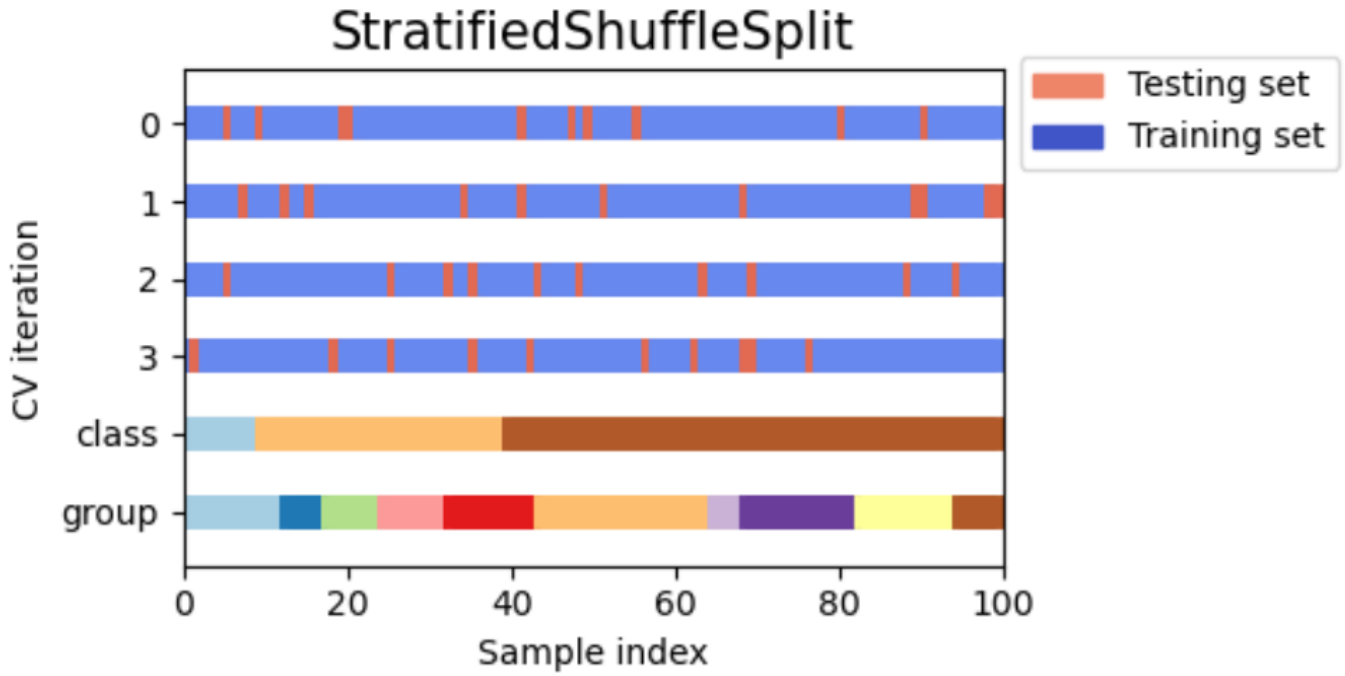
# Stratified k-Fold



# Stratified Shuffle Split

StratifiedShuffleSplit is a variation of ShuffleSplit, which returns stratified splits.

This creates splits by preserving the same percentage for each target class as in the complete set.



# Learnability – Weak vs Strong

- We can trade computational complexity with the requirement for accuracy.
- Given a distribution **D** and a target labeling function **f**, maybe there exists an efficiently computable learning algorithm whose error is just slightly better than a **random guess.**
- In **weak learnability**, we only need to output a hypothesis whose error rate is slightly better than what random labeling would give us.
- **Strong learnability** implies the ability to find an arbitrarily good classifier with an error rate at most for an arbitrarily small **ε** ($\varepsilon > 0$). (built on PAC framework)
- Hypothesis boosting was the idea of filtering observations, leaving those observations that the weak learner can handle and focusing on developing new weak learns to handle the remaining difficult observations.
- The immediate question is whether we can **boost** an efficient weak learner into an efficient strong learner.

# Boosting

- **Boosting** provides a tool for **aggregating weak hypotheses** to approximate gradually good predictors for larger, and harder to learn, classes.
- The learning starts with a basic class that might have a large **training** (approximation) **error**, and as it progresses the class that the predictor may belong to grows richer.
- The **boosting approach** uses a generalization of linear predictors to address the **Bias-Variance tradeoff**.
- The boosting paradigm allows the learner to have smooth control over this tradeoff.
- A **boosting algorithm amplifies the accuracy of weak learners**.

# AdaBoost

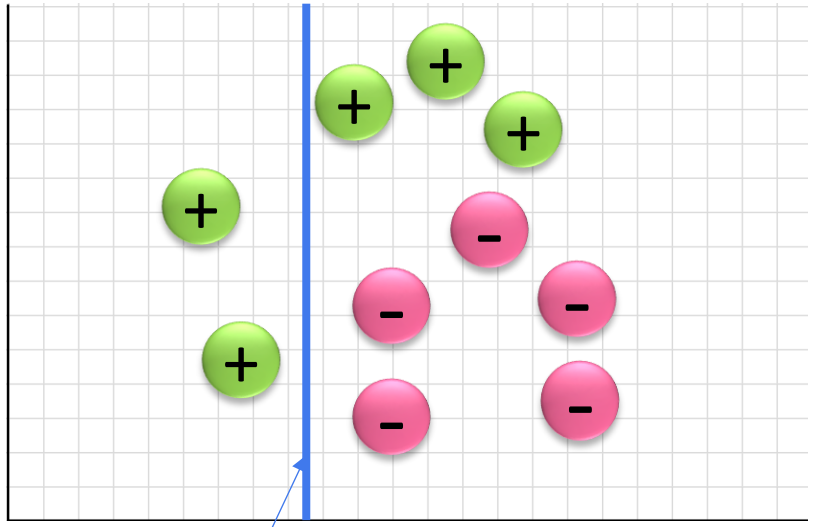
- AdaBoost stands for **Adaptive Boosting** and is a specific Boosting algorithm developed for **classification problems**.
- The **AdaBoost algorithm** outputs a **hypothesis** that is a **linear combination of simple hypotheses**.
- In other words, AdaBoost relies on the family of hypothesis classes obtained by composing a linear predictor on top of simple classes.
- In each **iteration**, AdaBoost identifies miss-classified data points, **increasing their weights**.
- It also **decreases the weights of correct points**, so that the next classifier will pay extra attention to get them right.

# AdaBoost

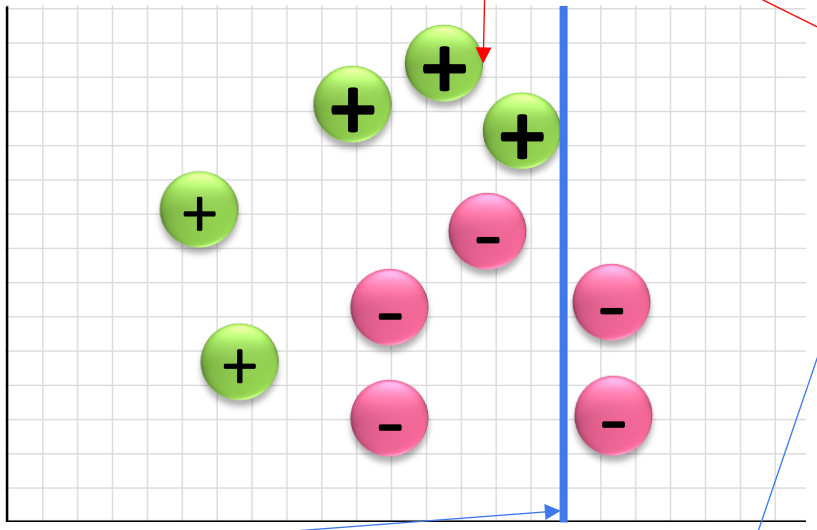
Weights Applied for wrongly classified data points

PACE UNIVERSITY SEIDENBERG SCHOOL OF CSIS

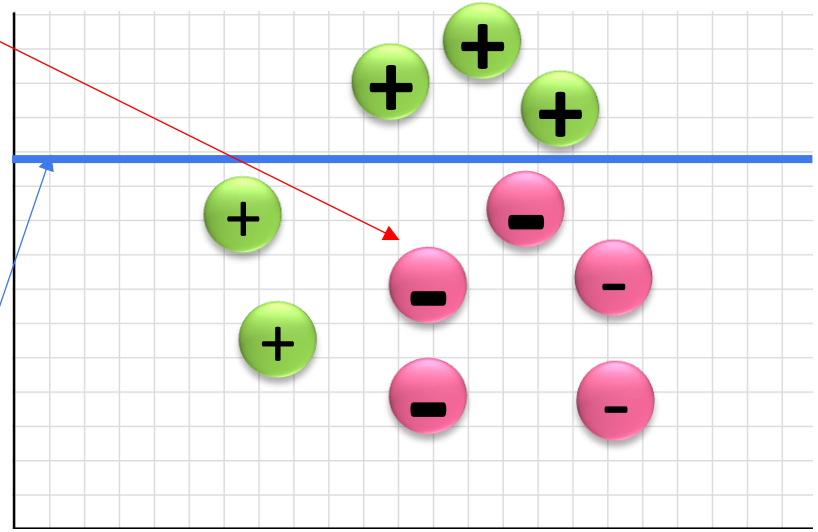
Model-1



Model-2

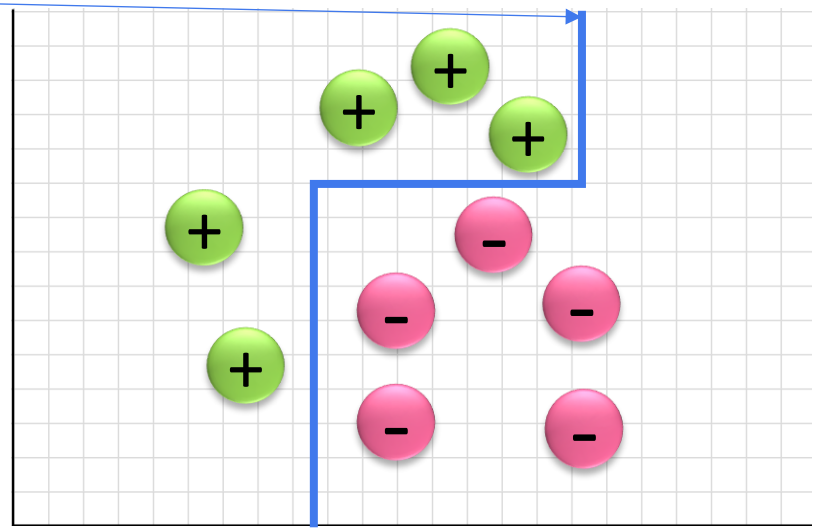


Model-3



Decision Boundary

Model-4



Model-4 = M1 + M2 + M3

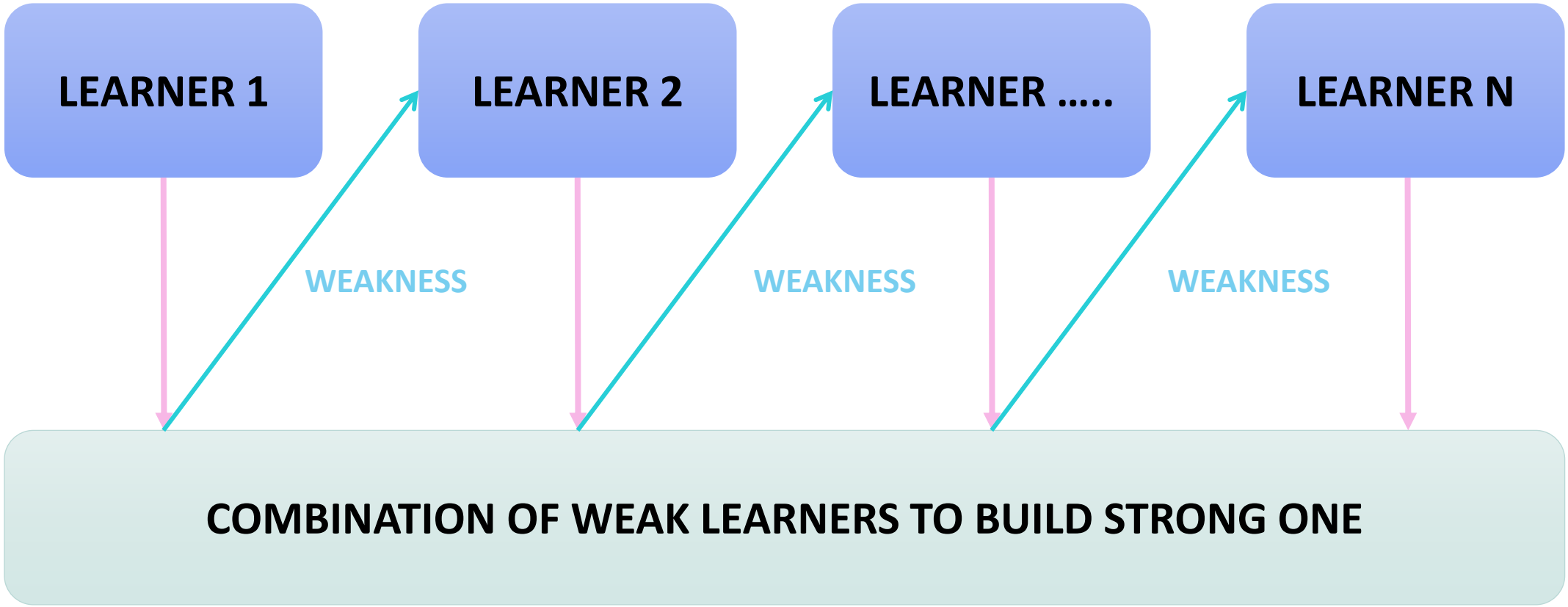
# AdaBoost

Now with the weakness defined, the next step is to figure out how to combine the sequence of models to make the final classifier stronger over time.

**Algorithm Steps**:

1. Initialize the dataset and assign equal weight to each data point.
2. With this as input to the model identify the wrongly classified data points.
3. Increase the weight of the wrongly classified data points.
4. if (got required results)
   Goto step 5
   else
   Goto step 2
5. End

# AdaBoost



# Gradient Boosting

**Gradient boosting** approaches the problem a bit differently.

Instead of adjusting the weights of data points, Gradient boosting focuses on the difference between the **prediction** and the **ground truth**.

Gradient boosting requires a **differential loss function** that works for both **regression** and **classification** tasks.

Each time we fit a **new estimator** to the gradient of loss. An estimator is a learning algorithm that trains on a Dataset(tabular) and produces a model

This class of algorithms were described as a **stage-wise additive model**.

This is because one **new weak learner is added at a time** and **existing weak learners in the model are frozen** and left unchanged.

# Gradient Boosting

**Gradient boosting** involves three elements:

1. A **loss function** to be optimized.
   For example, regression may use a squared error, and classification may use logarithmic loss.

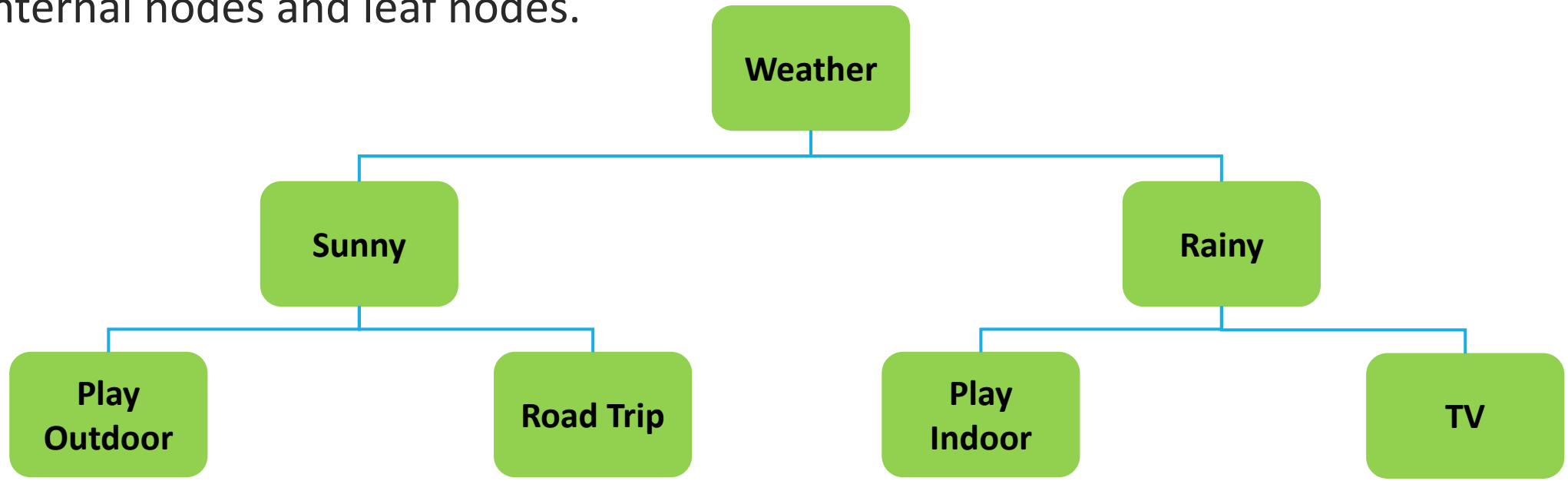
2. A **weak learner** in making predictions.
   **Decision trees** are used as the weak learner in **Gradient Boosting**. Trees are constructed in a greedy manner.

3. An **additive model** to add weak learners to minimize the loss function
   Trees are added one at a time, and existing trees in the model are not changed. A gradient descent procedure is used to minimize the loss when adding trees.

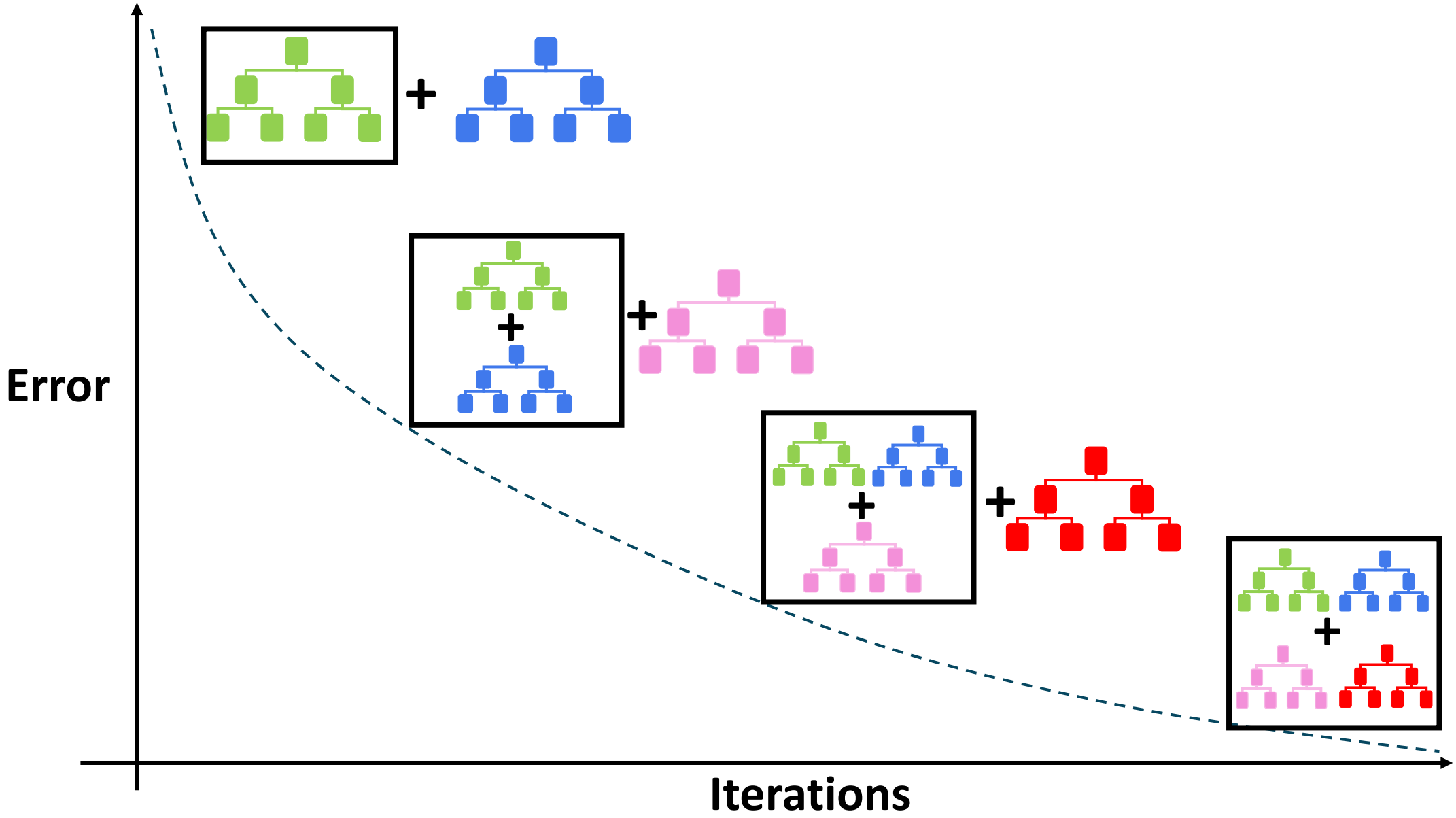
# Decision Trees

A **decision tree** is a non-parametric supervised learning algorithm, which is utilized for both **classification** and **regression** tasks.

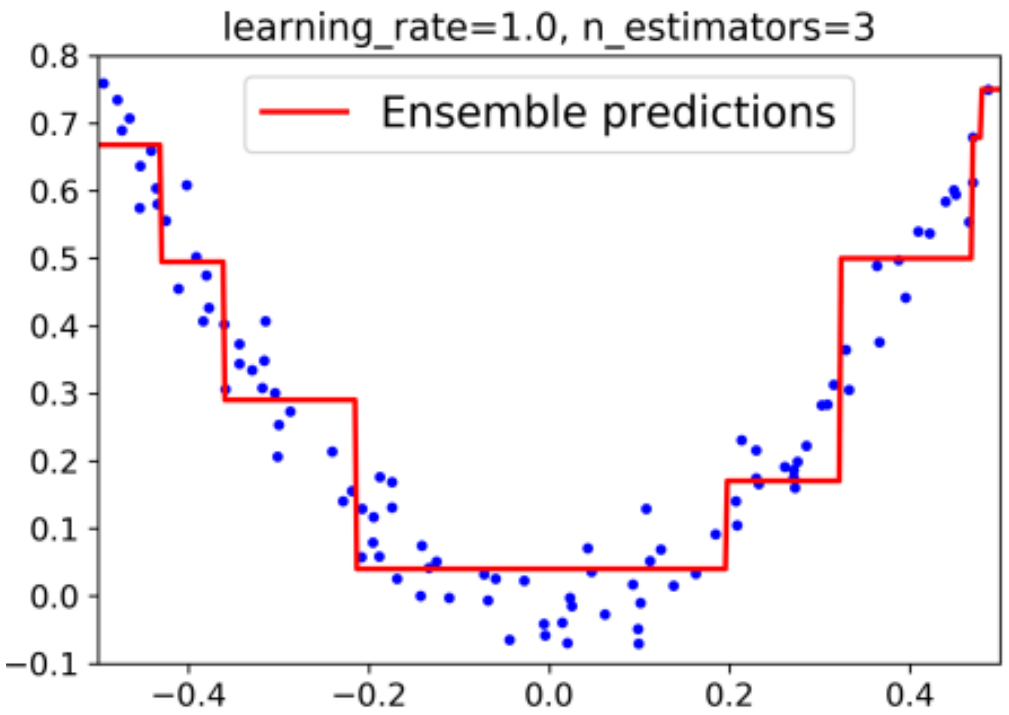
It has a hierarchical, tree structure, which consists of a root node, branches, internal nodes and leaf nodes.



# Gradient Boosting



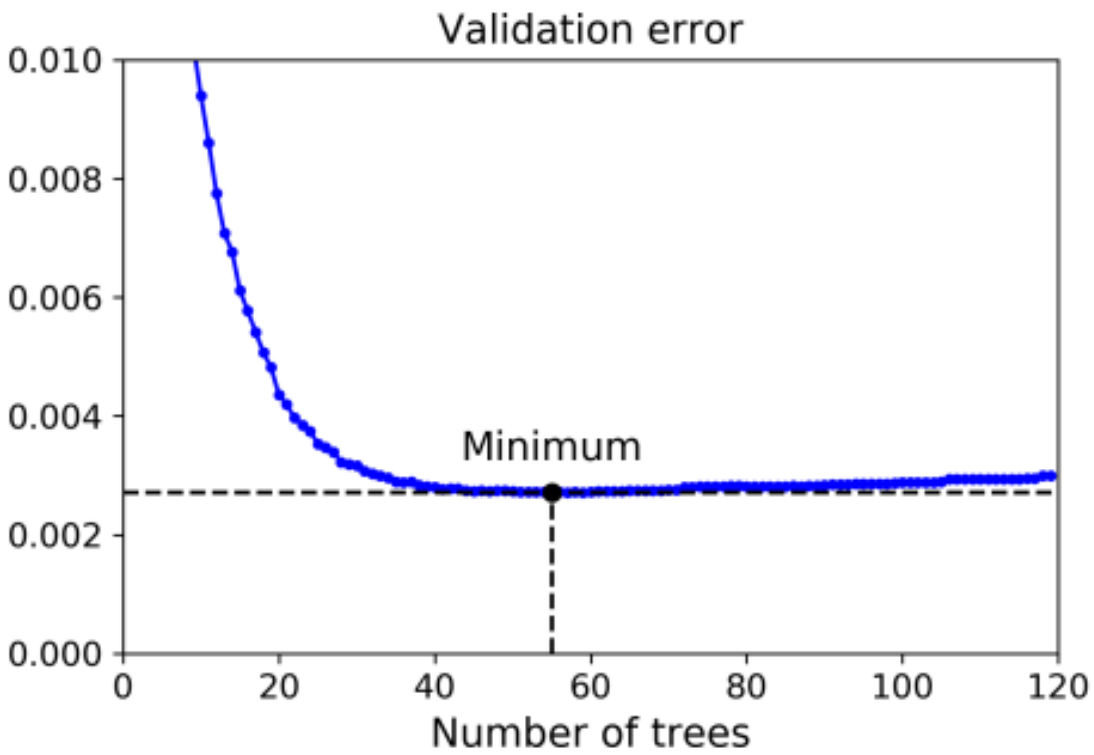
# Gradient Boosting

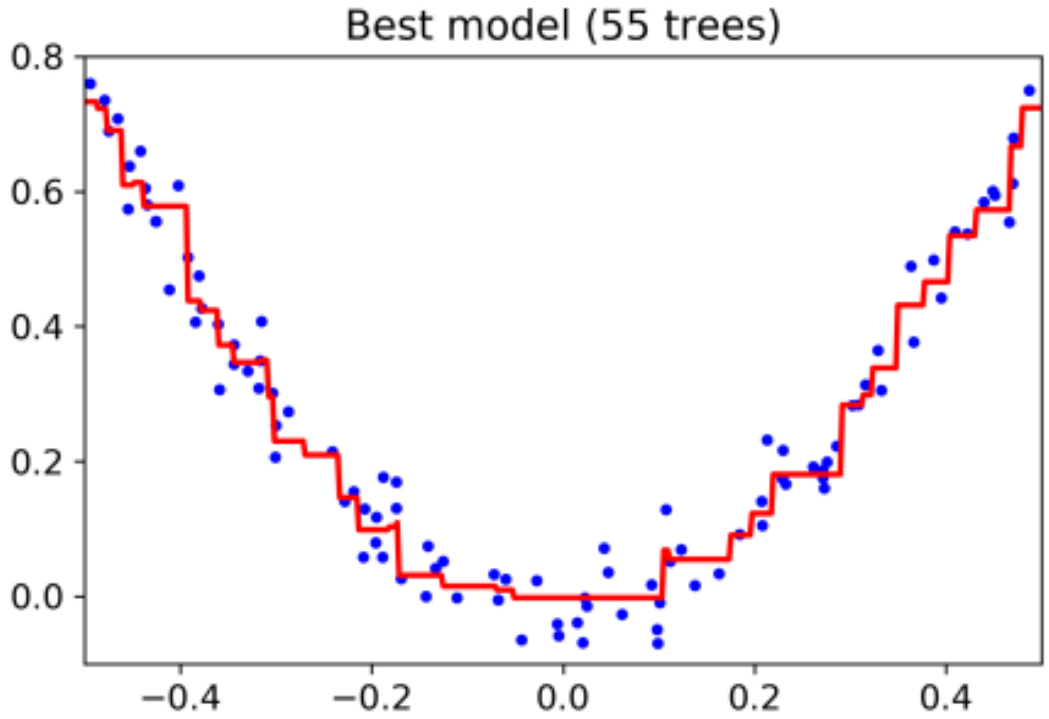



**Not enough estimators**


**Too many estimators**

# Gradient Boosting




**Early Stopping**

# END